

SOAP_BANK — Kubernetes Yolculuğu (Özet Referans)

Bu doküman, projeye Kubernetes + monitoring eklerken yaptığımız her şeyin özeti. Amaç, ileride "bunu nasıl yapmıştık" diye tekrar sorman gerekmesin.

1. Genel Mimari



5 servis, tek namespace ((soap-bank)) içinde: (account-service), (transaction-service), (api-gateway), (frontend), (redis), (postgres).

2. Docker İmajlarını Build Etme

Neden (eval \$(minikube docker-env)) kullanmadık: Windows'ta Git Bash ile çalışmadı. Bunun yerine imajları normal Docker Desktop'ta build edip, minikube'e elle "yükledik".

Build (her servis için aynı desen)

```
bash

docker build -f account-service.Dockerfile -t soap-bank/account-service:latest
docker build -f transaction-service.Dockerfile -t soap-bank/transaction-service:latest
docker build -f api-gateway.Dockerfile -t soap-bank/api-gateway:latest ./api-gateway
docker build -f frontend.Dockerfile -t soap-bank/frontend:latest ./frontend
```

Minikube'e Yükleme ((minikube image load) yerine bu yöntemi kullandık)

(minikube image load) Windows 11'de (wmic) eksikliği yüzünden çalışmadı. Çözüm: (save) → (cp) → (load).

```
bash
```

```
docker save -o <servis>.tar soap-bank/<servis>:latest
MSYS_NO_PATHCONV=1 minikube cp <servis>.tar /home/docker/<servis>.tar
MSYS_NO_PATHCONV=1 minikube ssh -- docker load -i /home/docker/<servis>.tar
```

MSYS_NO_PATHCONV=1 neden gerekli: Git Bash, Unix tarzı yolları (`/home/docker/...`) otomatik olarak Windows yoluna çevirmeye çalışıyor. Bu ön ek o davranışı kapatıyor. Bu ön eki minikube ile ilgili her komutta (`cp`, `ssh`, `run`) kullanma alışkanlığı edin.

3. `wsimport` / Maven Build Sorunları (Her SOAP client servisinde tekrar etti)

`account-service`, `transaction-service`, `api-gateway` — her birinin `pom.xml`'inde `jaxws-maven-plugin` var, WSDL'den Java client kodu üretiyor. Docker build ortamı, build sırasında `localhost:8081` gibi adreslere erişemediği için (izole container) bu adım patlıyor.

Çözüm: `skip-wsimport` Maven profili + `build-helper-maven-plugin`

`pom.xml`'e (her servis için, `</build>`'den sonra):

```
xml
<profiles>
  <profile>
    <id>skip-wsimport</id>
    <build>
      <plugins>
        <plugin>
          <groupId>org.codehaus.mojo</groupId>
          <artifactId>jaxws-maven-plugin</artifactId>
          <executions>
            <execution>
              <id>wsimport-account-service</id>
              <phase>none</phase>
            </execution>
            <!-- api-gateway'de ikinci bir execution daha var (tran
              o da aynı şekilde <phase>none</phase> ile atlanmal
          </executions>
        </plugin>
      </plugins>
    </build>
  </profile>
</profiles>
```

Ve `<build><plugins>` içine (`wsimport` atlansa da derlemenin `src/generated/java`'yı görmesi için):

xml

```
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>build-helper-maven-plugin</artifactId>
  <version>3.6.0</version>
  <executions>
    <execution>
      <id>add-generated-source</id>
      <phase>generate-sources</phase>
      <goals><goal>add-source</goal></goals>
      <configuration>
        <sources><source>${project.basedir}/src/generated/java</source>
      </configuration>
    </execution>
  </executions>
</plugin>
```

Build Sırası (önemli!)

Docker imajı build etmeden önce, `src/generated/java`'nın lokalde **dolu** olması lazım (wsimport'un bir kere lokalde çalışmış olması gerekiyor):

bash

```
# İlgili WSDL kaynakları ayakta olmalı
cd account-service && mvn spring-boot:run           # Terminal 1
cd transaction-service && mvn spring-boot:run        # Terminal 2 (api-gateway içi

# Sonra sources'ı üret
cd transaction-service && mvn clean compile          # Terminal 3
cd api-gateway && mvn clean compile                  # (account VE transaction i
```

Docker build'inde (`Dockerfile` içinde):

dockerfile

```
RUN mvn -B clean package -DskipTests -Pskip-wsimport
```

4. Kritik Runtime Sorunları ve Çözümleri

Bunlar, "Docker build başarılı ama pod çöküyor / SOAP çağrısı patlıyor" tarzı sorunlardı.

Sorun	Sebeup	Çöz
ImagePullBackOff	İmajlar minikube'ün içinde yoktu	save
CrashLoopBackOff (account/transaction)	localhost:5433 Postgres bağlantısı cluster'da yok	Pos yön
web-application-type: none	application.yaml'da unutulmuş test ayarı	Sild
SOAP client localhost'a bağlanıyor	wsimport'un ürettiği Service sınıflarında WSDL adresi build-time'da gömülü kalıyor	Her ove zorl
transaction-service'e (8084) Connection refused	Endpoint.publish("http://localhost:8084/...") sadece container'ın loopback'ine (127.0.0.1) bind oluyordu	Put gele
Provider com.sun.xml.ws.spi.ProviderImpl not found	JAX-WS'in kendi thread pool thread'lerinde (Endpoint.publish içeriden SOAP client çağırınca) context classloader Spring Boot fat-jar'ının classloader'ına ayarlı değildi	SOA Thi
CORS hatası (Failed to fetch)	api-gateway'in CORS config'i sadece localhost:5173'e (Vite dev server) izin veriyordu, minikube'ün nginx'i farklı bir port/origin kullanıyordu	.a.
Maven \${x:default} syntax'ı çalışmıyor	Bu Spring'e özgü bir syntax, düz pom.xml içinde ya da @Value olmayan bir Java String literalinde işe yaramaz	por

5. Kubernetes Manifestlerini Uygulama Sırası

bash

```
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/01-redis.yaml
kubectl apply -f k8s/07-postgres.yaml
kubectl apply -f k8s/02-account-service.yaml
kubectl apply -f k8s/03-transaction-service.yaml
kubectl apply -f k8s/04-frontend.yaml
kubectl apply -f k8s/08-api-gateway.yaml
kubectl apply -f k8s/05-hpa.yaml
```

Kod/env var değişikliği sonrası tek bir servisi yeniden başlatmak için (imaj yeniden build edilip minikube'e yüklendikten sonra):

```
bash
```

```
kubectl apply -f k8s/<değişen-dosya>.yaml  
kubectl rollout restart deployment <servis-adı> -n soap-bank
```

6. HPA (Otomatik Ölçeklendirme) — Nasıl Çalışıyor, Nasıl Test Edildi

Ön koşul: `minikube addons enable metrics-server` (CPU verisini HPA'ya sağlıyor).

Mantık: `k8s/05-hpa.yaml`'da `averageUtilization: 60` tanımlı — pod'ların ortalama CPU kullanımını `%60`'ı geçince Kubernetes otomatik olarak yeni pod ekliyor (`maxReplicas: 5`'e kadar). Yük düşünce, biraz gecikmeyle (`cooldown`) pod sayısı tekrar azalıyor (`minReplicas: 1`'e kadar).

Canlı test için yük üretici (paralel istek atan basit bir pod):

```
bash
```

```
MSYS_NO_PATHCONV=1 kubectl run load-generator -n soap-bank --image=busybox --restart=Never -- /bin/sh -c "for i in 1 2 3 4 5 6 7 8; do (while true; do wget -q -O- http://a
```

(Tek bir paralel istek CPU'yu yeterince yormadı, 8 paralel döngü işe yaradı.)

İzleme:

```
bash
```

```
kubectl get hpa -n soap-bank -w
```

Gördüğümüz gerçek sonuç:

```
cpu: 3%/60%    REPLICAS: 1    (boşta)  
cpu: 449%/60%  REPLICAS: 1    (yük binmiş, HPA henüz tepki vermedi)  
cpu: 449%/60%  REPLICAS: 4    (HPA devreye girdi)  
cpu: 449%/60%  REPLICAS: 5    (maxReplicas tavanı)
```

Testi bitirme:

```
bash
```

```
kubectl delete pod load-generator -n soap-bank
```

7. Monitoring: Prometheus + Grafana

bash

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace

kubectl apply -f k8s/06-servicemonitor.yaml
```

Grafana'ya erişim:

bash

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

`http://localhost:3000` — kullanıcı: `admin`, şifre:

bash

```
kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-p
```

Hazır dashboard'lar (ekstra kurulum gerekmiyor, kube-prometheus-stack ile geliyor):

- **Compute Resources / Namespace (Pods)** → `soap-bank` namespace'ini seç, pod bazlı CPU/RAM

Kendi sorgunu yazmak istersen (Explore sekmesi, PromQL):

```
kube_horizontalpodautoscaler_status_current_replicas{namespace="soap-bank"}
```

Bu, HPA'nın replika sayısını zaman içinde çizgi grafik olarak gösterir.

8. Sık Kullanılan Komutlar (Hızlı Referans)

bash

```
# Cluster durumu
minikube status
minikube start
kubectl get nodes

# Pod durumu
kubectl get pods -n soap-bank
kubectl logs -l app=<servis-adı> -n soap-bank --tail=100
kubectl describe pod <pod-adı> -n soap-bank

# Bir servise dışarıdan (lokal makineden) erişim
kubectl port-forward -n soap-bank svc/<servis-adı> <lokal-port>:<servis-port>

# Frontend'i tarayıcıda aç
minikube service frontend -n soap-bank

# Env var'ları kontrol et
kubectl get deployment <servis-adı> -n soap-bank -o yaml | grep -A 20 "env:"

# Değişiklik sonrası yeniden başlatma
kubectl apply -f k8s/<dosya>.yaml
kubectl rollout restart deployment <servis-adı> -n soap-bank
```

Özet: Bu Sürecin Genel Dersi

En çok zaman kaybettiğimiz şey, "**localhost her zaman aynı yeri işaret etmez**" gerçeği idi. Lokal makinende `localhost:8081` senin bilgisayarındır; bir container içinde `localhost` sadece o container'ın kendisidir; `Endpoint.publish("http://localhost:...")` de "hangi ağ arayüzünü dinleyeceğim" anlamına gelir. Kubernetes'e geçerken bu üç farklı anlamı ayırt edip her birini doğru şekilde (servis adı, `0.0.0.0`, ya da gerçek `localhost`) ayarlamak, sorunların neredeyse tamamının kökeniydi.